AURVIZ / SOFTWARE DEVELOPMENT ENGINEER

SDE Hiring Assignment

Build a Personal AI Agent — Android Application

Objective: Build an Android-based personal AI agent that understands natural-language voice commands, remembers information, and autonomously executes real-world tasks on behalf of the user.

The goal is not to build a simple AI chatbot. Build a functional AI agent that can take actions.

Core Requirements

1. Voice-to-Action AI Agent

- Accept voice and text commands.
- Integrate an AI developer API for natural-language understanding, planning, and tool calling.
- You may use the free tier of the Google Gemini API, subject to its usage limits.
- Execute actions based on user intent.
- Display task execution status and results.

2. Personal Memory Layer

- Implement persistent memory that works across app restarts.
- Remember user preferences, important information, and previous interactions.
- Retrieve relevant memories when executing new commands.
- Allow users to view and delete saved memories.

3. Real-World Task Execution

- Email: Compose and send emails using a connected email account.
- Web Automation: Open websites, navigate pages, scroll, and extract information through a controlled browser interface.
- Scheduled Tasks: Support recurring tasks using cron-like instructions, such as "Every morning at 9 AM, check my important emails."
- Multi-Step Actions: Process commands requiring multiple operations.

Example: "Every morning at 9 AM, check my important emails and prepare a summary."

The agent should understand the request, create the necessary workflow, schedule execution, and report the outcome.

Technical Expectations

- Native Android development using Kotlin is preferred.
- You may use the free Gemini API or another suitable AI developer API.
- Design a modular architecture separating UI, agent orchestration, memory, tools, and scheduling.
- Handle permissions, authentication, network failures, background execution, and API errors.
- Store credentials securely. Never commit API keys or OAuth tokens to GitHub.
- Require explicit user confirmation before sensitive actions such as sending emails.

- Ensure scheduled tasks remain functional within Android’s background-execution constraints.

You are free to select appropriate libraries, frameworks, databases, and architectural patterns.

Submission Requirements

1. GitHub Repository

Upload your complete project to GitHub and provide:

- Repository URL
- README with setup and execution instructions
- APK or reproducible build instructions
- Sample environment configuration without secrets

2. Architecture Documentation

Include an architecture diagram and explain:

- Overall system architecture
- AI agent reasoning and tool-calling workflow
- Memory storage and retrieval design
- Voice-to-action execution pipeline
- Scheduling and background execution strategy
- Security, permissions, and failure handling
- Important engineering decisions and trade-offs

3. Performance Analysis

Provide basic measurements of agent response time, task execution latency, and memory usage. Explain at least one performance improvement or optimization opportunity.

4. AI Development Tools

You may use Codex, Claude Code, ChatGPT, Cursor, or any other AI development tool.

We will evaluate your ability to understand, direct, validate, and improve AI-generated implementations—not the quantity of code you write manually.

5. Cost Requirements

Candidates are not required to purchase any paid API, subscription, or service. The assignment can be completed using free development tools and available API free tiers. Paid services will not receive additional evaluation credit.

Evaluation Criteria

Category	Weight
Agent reasoning and action execution	25%
System architecture and problem-solving	25%
Memory and scheduling reliability	20%
Code quality, security, and error handling	15%
Performance, testing, and documentation	15%

Submission by Email

Subject: Aurviz SDE Assignment Submission – [Candidate Name]

Email your submission with:

- GitHub repository link
- Architecture diagram and explanation
- Setup instructions
- Completed and incomplete features
- Key technical challenges and solutions

Deadline: 13 November 2026